

자료구조 실습 보고서

[제 10 주] 연결체인 큐

제출일 : 2018 년 5 월 12 일

201702081 최재범

1. 프로그램 설명서

a. 주요 알고리즘 / 자료구조

지난번에 Queue 를 배열을 이용하여 구현하였던 것을, 이번에는 연결 체인을 이용하여 구현하였다. 자료를 저장하는 내부 구조만을 변경한 것이기 때문에, AppController 등의 구현은 거의 바뀌지 않는다.

연결 체인에서는 각 노드에 다음 원소에 대한 정보를 가지고 있으며, 환형으로 구현하기 때문에 맨 뒤의 원소는 항상 다음 원소로 맨 앞의 원소를 가지게 된다. 따라서 front 를 나타내는 필드는 필요하지 않다. 하지만 배열은 순번만 매겨질 뿐 맨 앞 원소와 맨 뒤 원소 간에 직접적인 관계가 없기 때문에, front 에 대한 접근을 위하여 따로 필드가 필요하다.

추가, 제거를 반복하다 보면 front 가 점점 앞으로 향하는 현상이 연결 체인을 이용하면 사라진다. 이는 배열은 크기를 정함에 따라 처음과 마지막 인덱스 번호가 정해지기 때문에 원소마다 순번이 매겨지지만, 연결 체인은 크기가 메모리가 허용하는 한 무한하며 순번도 매겨지지 않기 때문이다.

환형을 구현하기 위해, 배열과 연결 체인 모두 추가, 제거 과정에서 처음과 끝을 잇는 과정을 포함하였다. 다만 연결 체인에서는 각 노드가 다음 원소에 대한 위치 정보를 갖고있기 때문에 그냥 Getter 와 Setter 을 이용하여 설정해주면 되었지만, 배열에서는 그런 것이 없기 때문에 인덱스를 증가시킨 뒤에 조건문을 이용하여 인덱스 범위 초과 여부를 확인 뒤, 초과하였으면 맨 앞의 인덱스를 다시 대입하는 식으로 하였다.

- public boolean enqueue(T anElement) : 원소 추가

Queue 가 비어있을 때에는 첫 번째 원소를 넣게 되는데,

환형 큐에서는 맨 끝 노드가 맨 처음을 가리켜야 하기 때문에 첫 번째 원소의 노드는 다음 노드로 자기 자신을 가리키도록 한다.

그 후 두 번째 노드부터는 맨 뒤에 추가되므로, 다음 노드를 맨 앞의 노드로 하고 맨 뒤의 노드는 자신을 가리키도록 한다. 그 후 자신을 맨 뒤의 노드로 설정한다.

그 후 크기 필드를 증가시킨다.

- public T dequeue() : 원소 제거

Queue 에 남아있는 노드 수가 1 개일 때는 맨 뒤의 노드를 가리키는 rear 필드를 null 로 초기화 시킨다.

2 개 이상 남았을 때는, 삭제할 맨 앞 원소 (맨 뒤의 next) 를 반환을 위해 저장하고 나서 맨 앞에서 두 번째 노드를 맨 뒤 노드가 가리키게 한다. 기존 맨 앞의 원소는 아무에게도 가리켜지지 않아 가비지 콜렉터에 의하여 정리된다.

그 후 크기 필드를 감소시키고, 저장해놓았던 원소를 반환한다.

b. 함수 설명서

■ ApplicationController

- public void run() : 프로그램 실행

■ AppView

- public int inputInt() : 정수 입력받아 반환
- public String inputString() : 문자열 입력받아 반환
- public char inputCharacter() : 문자 1 개 입력받아 반환
- public void outputAdd(char anElement) : 추가된 원소 출력
- public void outputElement(char anElement) : Queue 원소 하나 출력
- public void outputFrontElement(char anElement) : Front 원소 출력
- public void outputQueueSize(int aStackSize) : Queue 의 현재 크기 출력
- public void outputRemove(char anElement) : 제거된 원소 출력
- public void outputRemoveN(int numberOfCharsToBeRemoved) : 제거될 원소의 수 출력
- public void outputResult(int numberOfInputChars, int numberOfIgnoredChars, int numberOfAddedChars) : 입력 처리 결과 출력
- public void outputMessage(String aMessageString) : 문자열 출력

■ CircularArrayQueue

- public CircularlyLinkedQueue() : rear 과 size 초기화
- public int capacity() : 최대 크기 반환 (Getter)
- public int size() : 현재 크기 반환 (Getter)
- public boolean isEmpty() : 비어있는지 확인
- public boolean isFull() : 꽉 찼는지 확인
- public boolean enqueue(T anElement) : 원소 추가
- public T dequeue() : 원소 제거 후 반환
- public T frontElement() : 맨 앞의 원소 확인 후 반환
- public void clear() : Queue 초기화
- public T elementAt(int anOrder) : 주어진 순번의 원소 반환

■ Node

- public Node() : 디폴트 생성자
- public Node(T givenElement) : 노드에 담을 원소만 설정하고, 다음 노드의 위치 정보는 설정하지 않음 (생성자)
- public Node(T givenElement, Node<T> givenNode) : 노드에 담을 정보와 다음 노드의 위치 정보 설정 (생성자)
- public T element() : 노드에 담긴 원소 반환 (Getter)
- public Node<T> next() : 다음 노드의 위치 정보 반환 (Getter)

- public void setElement(T newElement) : 노드에 원소 설정 (Setter)
- public void setNext(Node<T> newNext) : 노드에 다음 노드의 위치 정보 설정 (Setter)

c. 종합 설명서

A 부터 Z, a 부터 z 까지의 알파벳을 입력받으면 Queue 에 추가한다.

0 부터 9 까지의 숫자를 넣으면, 해당 숫자만큼 Queue 에서 제거한다.

- 를 넣으면, Queue 에서 하나를 제거한다.

를 넣으면, 현재 Queue 의 크기를 출력한다.

/ 를 넣으면, Queue 의 상태를 Front 부터 시작하여 Rear 까지 출력한다.

^ 를 넣으면, Front 에 위치한 원소를 출력한다.

! 를 넣으면, 스택 원소를 전부 제거하고 입력에 대한 처리 여부를 출력 후 종료 메시지를 출력한다.

이외의 입력이 들어올 시, 오류를 출력하고 무시한다.

2. 프로그램 장단점 분석

장점 : 배열로 구현한 것과 달리, 크기가 메모리가 허용하는 한 무한하다.

단점 : 문자를 입력할 때 아무것도 입력하지 않고 엔터를 치면 예외가 발생한다.

3. 실행 결과 분석

a. 입력과 출력

> 프로그램을 시작합니다.
[큐 사용을 시작합니다.]

- 문자를 입력하시오 : a
[EnQueue] 삽입된 원소는 'a' 입니다.

- 문자를 입력하시오 : b
[EnQueue] 삽입된 원소는 'b' 입니다.

- 문자를 입력하시오 : c
[EnQueue] 삽입된 원소는 'c' 입니다.

- 문자를 입력하시오 : d
[EnQueue] 삽입된 원소는 'd' 입니다.

- 문자를 입력하시오 : E
[EnQueue] 삽입된 원소는 'E' 입니다.

- 문자를 입력하시오 : 3
[RemoveN] 3 개의 원소를 삭제하려고 합니다.
[DeQueue] 삭제된 원소는 'a' 입니다.
[DeQueue] 삭제된 원소는 'b' 입니다.
[DeQueue] 삭제된 원소는 'c' 입니다.

- 문자를 입력하시오 : -
[DeQueue] 삭제된 원소는 'd' 입니다.

- 문자를 입력하시오 : #
[Size] 큐에는 현재 1 개의 원소가 있습니다.

- 문자를 입력하시오 : /
[Queue] <Front> E <Rear>

- 문자를 입력하시오 : ^
[Front] Front 원소는 'E' 입니다.

- 문자를 입력하시오 : %
[Error] 의미 없는 문자가 입력되었습니다.

- 문자를 입력하시오 : rk
[EnQueue] 삽입된 원소는 'r' 입니다.

- 문자를 입력하시오 : !
[큐 입력을 종료합니다.]
[DeQueue] 삭제된 원소는 'E' 입니다.
[DeQueue] 삭제된 원소는 'r' 입니다.
입력된 문자는 모두 12 개 입니다.
추가된 문자는 모두 6 개 입니다.
무시된 문자는 모두 1 개 입니다.

> 프로그램을 종료합니다.

b. 결과분석

a, b, c, d, E 의 총 5 개 원소를 추가하였다.

3 개를 제거할 때 먼저 추가한 a, b, c 가 삭제되었고, 그 후에 -를 하니 Front 에 있는 원소 d 가 제거되었다.

크기를 확인하니 E 만 남아 1 개라고 출력되었으며, 전체 형태를 출력하니 E 가 나왔고 Front 를 출력해도 E 가 나왔다.

의미 없는 문자인 %를 입력하니 무시되었다.

한 번에 알파벳 하나만 받아들일 수 있기 때문에, rk 를 입력하였더니 맨 앞의 r 만 추가되었다.

입력된 문자 : a, b, c, d, E, 3, -, #, /, ^, %, r : 12 개 (! 는 제외)

추가된 문자 : a, b, c, d, E, r : 6 개

무시된 문자 : % : 1 개

따라서 맞는 결과가 나왔다고 할 수 있다.

4. 소스코드

AppController.java

```
public class AppController
{
    private AppView _appView;
    private CircularlyLinkedList <Character> _queue;
    private int _inputChars;           // 입력된 문자의 개수
    private int _ignoredChars;        // 무시된 문자의 개수
    private int _addedChars;          // 삽입된 문자의 개수

    // 생성자
    public AppController() {
        this._appView = new AppView();
        this.initCharCounts();
    }

    // 비공개 함수
    // 출력 관련

    // 맨 위 원소 출력
    private void showFrontElement() {
        if(this._queue.isEmpty()) {
            this._appView.outputMessage("[Empty] 큐가 비어있습니다. \n");
        }
        else {
            char frontElement = (char)this._queue.frontElement();
            this._appView.outputFrontElement(frontElement);
        }
    }
}
```

```

}

// Queue 크기 출력
private void showQueueSize() {
    int size = this._queue.size();
    this._appView.outputQueueSize(size);
}

// Front 부터 전체 출력
private void showAll() {

    this._appView.outputMessage("[Queue] <Front>");

    int size = this._queue.size();
    for(int order = 0; order < size; order++) {
        this._appView.outputElement(this._queue.elementAt(order));
    }

    this._appView.outputMessage(" <Rear>\n");
}

// 횃수 계산 관련
// 횃수 초기화
private void initCharCounts() {
    this._inputChars = 0;
    this._ignoredChars = 0;
    this._addedChars = 0;
}

// 추가 / 무시 / 입력된 원소 수 셈
private void countAdded() {
    this._addedChars++;
}
private void countIgnored() {
    this._ignoredChars++;
}
private void countInputChar() {
    this._inputChars++;
}

// 스택 조작 관련
// 추가
private void add(Character anElement) {
    if(this._queue.enqueue(anElement)) {
        this._appView.outputAdd((char)anElement);
        this.countAdded();
    }
    else {
        this.showMessage(MessageID.Error_InputFull);
    }
}

// 1개 제거 후 출력
private void removeOne() {
    if(this._queue.isEmpty()) {
        this.showMessage(MessageID.Error_Empty);
    }
    else {
        Character removedElement = this._queue.dequeue();
        this._appView.outputRemove(removedElement);
    }
}

```

```

    }
}

// 입력된 개수만큼 제거 후 출력
private void removeN(int numberOfCharsToBeRemoved) {
    this._appView.outputRemoveN(numberOfCharsToBeRemoved);
    for(int i = 0; i < numberOfCharsToBeRemoved; i++) {
        this.removeOne();
    }
}

// 전부 제거 후 출력
private void conclusion() {
    int size = this._queue.size();
    for(int i = 0; i < size; i++) {
        this.removeOne();
    }
    this._appView.outputResult(this._inputChars, this._ignoredChars, this._addedChars);
}

// 상황에 맞는 메시지 출력
private void showMessage(MessageID aMessageID) {
    switch(aMessageID) {

        case Notice_StartProgram :
            this._appView.outputMessage("> 프로그램을 시작합니다. \n");
            break;

        case Notice_EndProgram :
            this._appView.outputMessage("> 프로그램을 종료합니다. \n");
            break;

        case Notice_StartMenu :
            this._appView.outputMessage("[ 큐 사용을 시작합니다. ] \n");
            break;

        case Notice_EndMenu :
            this._appView.outputMessage("[ 큐 입력을 종료합니다. ] \n");
            break;

        case Error_WrongMenu :
            this._appView.outputMessage("[Error] 의미 없는 문자가 입력되었습니다. \n");
            break;

        case Error_InputFull :
            this._appView.outputMessage("[Full] 큐가 꽉 차서 삽입이 불가능합니다. \n");
            break;

        case Error_Empty :
            this._appView.outputMessage("[Empty] 큐에 삭제할 원소가 없습니다. \n");
            break;

    }
}

// 공개 함수
public void run() {

    this._queue = new CircularlyLinkedListQueue<Character>();

```

```

        char inputChar;                // 메뉴 입력 저장

        this.showMessage(MessageID.Notice_StartProgram);
        this.showMessage(MessageID.Notice_StartMenu);

        inputChar = this._appView.inputCharacter();
        while(inputChar != '!') {

            this.countInputChar();
            if( (inputChar >= 'a' && inputChar <= 'z') ||
                (inputChar >= 'A' && inputChar <= 'Z') )
            {
                this.add(inputChar);
            }

            else if(inputChar >= '0' && inputChar <= '9') {
                this.removeN( Integer.parseInt( String.valueOf(inputChar) ) );
            }
            else if(inputChar == '-') {
                this.removeOne();
            }
            else if(inputChar == '#') {
                this.showQueueSize();
            }
            else if(inputChar == '/') {
                this.showAll();
            }
            else if(inputChar == '^') {
                this.showFrontElement();
            }
            else {
                this.showMessage(MessageID.Error_WrongMenu);
                this.countIgnored();
            }

            inputChar = this._appView.inputCharacter();        // 새로운 입력
        }

        this.showMessage(MessageID.Notice_EndMenu);
        this.conclusion();
        this.showMessage(MessageID.Notice_EndProgram);

    }
}

```

AppView.java

```

import java.util.Scanner;

public class AppView {

    private Scanner _scanner;

    // 생성자
    public AppView() {
        this._scanner = new Scanner(System.in);
    }

    // 공개 함수

    // 입력 관련
    // 정수 입력받아 반환
    public int inputInt() {
        return Integer.parseInt(this._scanner.nextLine());
    }
}

```



```

}

// String 입력받아 반환
public String inputString() {
    return this._scanner.nextLine();
}

// char 입력받아 반환
public char inputCharacter() {
    char element;
    System.out.print("\n- 문자를 입력하시오 : ");
    element = this.inputString().charAt(0);
    return element;
}

// 출력 관련
// 추가된 원소 출력
public void outputAdd(char anElement) {
    System.out.printf("[EnQueue] 삽입된 원소는 '%c' 입니다. \n", anElement);
}

// Queue 원소 하나 출력
public void outputElement(char anElement) {
    System.out.printf("%3c", anElement);
}

// Front 원소 출력
public void outputFrontElement(char anElement) {
    System.out.printf("[Front] Front 원소는 '%c' 입니다. \n", anElement);
}

// Queue의 현재 크기 출력
public void outputQueueSize(int aQueueSize) {
    System.out.printf("[Size] 큐에는 현재 %d 개의 원소가 있습니다. \n", aQueueSize);
}

// 제거된 원소 출력
public void outputRemove(char anElement) {
    System.out.printf("[DeQueue] 삭제된 원소는 '%c' 입니다. \n", anElement);
}

// 제거될 원소의 수 출력
public void outputRemoveN(int numberOfCharsToBeRemoved) {
    System.out.printf("[RemoveN] %d 개의 원소를 삭제하려고 합니다. \n", numberOfCharsToBeRemoved);
}

// 입력 처리 결과 출력
public void outputResult(int numberOfInputChars,
                        int numberOfIgnoredChars,
                        int numberOfAddedChars) {
    System.out.printf("입력된 문자는 모두 %d 개 입니다. \n", numberOfInputChars);
    System.out.printf("추가된 문자는 모두 %d 개 입니다. \n", numberOfAddedChars);
    System.out.printf("무시된 문자는 모두 %d 개 입니다. \n", numberOfIgnoredChars);
}

// String 출력
public void outputMessage(String aMessageString) {
    System.out.print(aMessageString);
}

```

```
}
```

CircularlyLinkedList

```
public class CircularlyLinkedList<T> implements Queue<T>
{

    private int _size;
    private Node<T> _rear;

    // 생성자
    public CircularlyLinkedList() {
        this._size = 0;
        this._rear = null;
    }

    // 공개 함수

    // 최대 크기 반환 : Getter
    public int capacity() {
        return Integer.MAX_VALUE;
    }

    // 현재 크기 반환 : Getter
    public int size() {
        return this._size;
    }

    // 비어있는지 확인
    public boolean isEmpty() {
        return this._rear == null;
    }

    // 꽉 찼는지 확인
    public boolean isFull() {
        return false;
    }

    // 원소 추가 : 연결 체인 이므로 가득 찰 경우 없음
    public boolean enqueue(T anElement) {
        if(this.isEmpty()) {
            Node<T> newNode = new Node<T>(anElement);
            newNode.setNext(newNode); // 맨 처음에 추가하는 원소는 next가 자기 자신을 가리키게 됨
            this._rear = newNode;
        }
        else {
            // rear에 추가할 노드는 front의 노드를 가리키게 됨
            Node<T> newNode = new Node<T>(anElement, this._rear.next());
            this._rear.setNext(newNode);
            this._rear = newNode;
        }
        this._size++;
        return true;
    }

    // 원소 제거 후 반환
}
```

```

public T deQueue() {

    // 노드가 없을 때
    if(this.isEmpty()) {
        return null;
    }

    // 노드 수가 1 개일 때
    else if(this.size() == 1) {
        T removedElement = this._rear.element();
        this._rear = null;
        this._size--;
        return removedElement;
    }

    // 노드 수가 2 개 이상일 때
    else {
        T removedElement = this._rear.next().element();
        this._rear.setNext(this._rear.next().next());
        this._size--;
        return removedElement;
    }
}

// front 의 원소 확인 후 반환 : 환형이므로 rear의 다음 노드가 front
public T frontElement() {
    if(this.isEmpty()) {
        return null;
    }
    else {
        return this._rear.next().element();
    }
}

// Queue 초기화
public void clear() {
    this._size = 0;
    this._rear = null;
}

// 주어진 순서의 원소 반환
public T elementAt(int anOrder) {
    if(0 <= anOrder && anOrder <= this._size - 1) {
        Node<T> front = this._rear.next();

        Node<T> orderNode = front;
        for(int order = 0; order < anOrder; order++) {
            orderNode = orderNode.next();
        }

        return orderNode.element();
    }
    else {
        return null;
    }
}
}

```

Node.java

```
public class Node<T>
```

```

{

    private T _element;
    private Node<T> _next;

    // Constructor
    public Node() {}
    public Node(T givenElement) {
        this._element = givenElement;
        this._next = null;
    }
    public Node(T givenElement, Node<T> givenNode) {
        this._element = givenElement;
        this._next = givenNode;
    }

    // Getter
    public T element() {
        return this._element;
    }
    public Node<T> next() {
        return this._next;
    }

    // Setter
    public void setElement(T newElement) {
        this._element = newElement;
    }
    public void setNext(Node<T> newNext) {
        this._next = newNext;
    }

}

```

Queue.java

```

public interface Queue<T>
{

    public boolean isEmpty();
    public boolean isFull();
    public boolean enqueue(T anElement);
    public T dequeue();
    public T frontElement();

}

```

MessageID.java

```

public enum MessageID {

    // Message IDs for Notices :
    Notice_StartProgram,
    Notice_EndProgram,
    Notice_StartMenu,
    Notice_EndMenu,

    // Message IDs for Errors :
    Error_WrongMenu,
    Error_InputFull,
    Error_Empty,

}

```

DS1_10_201702081_최재범.java

```
public class DS1_10_201702081_최재범 {  
  
    public static void main(String[] args) {  
  
        AppController appController = new AppController();  
        appController.run();  
  
    }  
  
}
```